Chapter 6. Designing lock-based concurrent data structures

This chapter covers

- What it means to design data structures for concurrency
- Guidelines for doing so
- Example implementations of data structures designed for concurrency

In the last chapter we looked at the low-level details of atomic operations and the memory model. In this chapter we'll take a break from the low-level details (although we'll need them for chapter 7) and think about data structures.

The choice of data structure to use for a programming problem can be a key part of the overall solution, and parallel programming problems are no exception. If a data structure is to be accessed from multiple threads, either it must be completely immutable so the data never changes and no synchronization is necessary, or the program must be designed to ensure that changes are correctly synchronized between threads. One option is to use a separate mutex and external locking to protect the data, using the techniques we looked at in chapters 3 and 4, and another is to design the data structure itself for concurrent access.

When designing a data structure for concurrency, you can use the basic building blocks of multithreaded applications from earlier chapters, such as mutexes and condition variables. Indeed, you've already seen a couple of examples showing how to combine these building blocks to write data structures that are safe for concurrent access from multiple threads.

In this chapter we'll start by looking at some general guidelines for designing data structures for concurrency. We'll then take the basic building